

Automate your builds with GitHub Actions

Source code [here](#)

Demonstration video [here](#)

Continuous Integration and Deployment (CI/CD) pipelines are an essential part of streamlining software development workflows, ensuring top-notch code quality, and delivering updates with speed and accuracy.

In this article, you'll learn how to publish automatically Docker images to a registry, such as Docker Hub with GitHub Actions, to then deploy to the target container, as part of your (CI/CD) workflow.

Author of the article: *Iker Elg. Alzaa*

#DockerActions #ContinuousIntegration #ContinuousDeployment



Embracing the tools that we will describe here, allows you to focus more on coding and less on

deployment intricacies, unlocking the potential to transform your software deployment process with the integration of GitHub Actions, Docker Compose, and Watchtower implementing a powerful, automated, and efficient CI/CD pipeline for your projects.

We will be dockerizing a simple backend Web API service and setting up the pipeline for it. It's worth mentioning that although this solution may not be ideal for larger applications, it's still a highly effective method for rapidly setting up CI/CD pipelines for less complex applications or testing environments.

Setting up Docker Hub

We will need a Docker Hub account and container registry or repository to setup our pipelines and store our images. For that, in the Docker Hub dashboard, click on create repository button which you can find on the top right side below the navigation bar. For ore information check out the link below.

- [kraftcoding/githubactionsci-cdexamplewebapinet-general](https://kraftcoding.github.io/actions/ci-cd/example/webapi-net-8/dockerfile)

We will also need access token to be able to push our docker images from github actions. To generate an access token head over to Settings > Personal Access Token > Generate New Token.

Setting up Github Actions

Now, let's add a Github Actions workflow to the repository. We will need a GitHub account for that.

In the local project with the Visual Studio or any other IDE, create a directory named `.github` in the root of the solution folder and inside that directory create another directory named `workflows`. All our workflow files will be stored inside this directory.

Now, create a file named `deploy-to-docker-hub.yml` which will make the complete path as `.github/workflows/deploy-to-docker-hub.yml`.

```

deploy-to-docker-hub.yml
jobs:
  push_to_registry:
    name: Push Docker image to Docker Hub
    runs-on: ubuntu-latest
    permissions:
      packages: write
      contents: read
      attestations: write
      id-token: write
    steps:
      - name: Check out the repo
        uses: actions/checkout@v5

      - name: Log in to Docker Hub
        uses: docker/login-action@f4ef78c808cd8ba55a85445d5b36214a81df20a
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: Extract metadata (tags, labels) for Docker
        id: meta
        uses: docker/metadata-action@9ec57ed1fcd14dcf7dfe97b2010124a938b7
        with:
          images: kraftcoding/githubactionsci-cdexamplewebapinet #automatically the gih

      - name: Build and push Docker image
        id: push
        uses: docker/build-push-action@3b5e8027fcad23fda98b2e3ac259d8d67585f671
        with:
          context: .
          file: ./Github-Actions-CI-CD-Example-WebAPI-.NET-8/Dockerfile
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}

      - name: Generate artifact attestation
        uses: actions/attest-build-provenance@v3
        with:
          subject-name: index.docker.io/kraftcoding/githubactionsci-cdexamplewebapinet
          subject-digest: ${{ steps.push.outputs.digest }}
          push-to-registry: true
  
```

As we can see in the source code provided, we have also a *Dockerfile* reference in our project directory with which we can build our docker image.

In addition we will need too, the *Docker Composite* file called `docker-compose.yml` to define the container's features that will be created from the image.

Once we have all the need files, use your Docker Hub username and password created earlier to setup the secret variables in your GitHub repository Settings > Security > Secrets and variables > Actions configuration section.

Now, whenever we push to *master* branch, a new build will be triggered which will build and push the image to docker hub. The workflow is pretty simple and fast to execute and you may also extend it with running tests or performing other actions as per your requirements.

Initial deploy

In order to have a running container based in the image together with the *Watchtower* agent that will check for updates in the *Docker Hub* repository (that we pushed earlier to *Docker Hub*), execute the following command:

```
docker compose up -d
```

As we can see, we are also pulling the watchtower image and running it in our machine.

This watchtower container will scan the repository of specified container names every 30 seconds pull and replace the running container with a new image if it finds one.

Run the command below to view the logs of watchtower container:

```
docker logs watchtower
```

We can see from the logs that watchtower has started scanning our repository for new images.

In cases of private repositories, watchtower will need additional configuration to scan repo for new images.